MISSION ARES IV

Take-Home Final Exam

Elements of Computing Systems • Ben-Gurion University

Submission instructions

Submit a single ZIP file named **`<YourID>.zip`** containing exactly two files:

- **`part1.pdf`** — your handwritten or typed solution to Part 1 (scanned or exported as PDF)
- **`student_code.asm`** — your completed MIPS assembly file for Part 2

Do not include `acpu_stub.asm` or the scaffold in your submission — only your own work.

Academic integrity

You may use any reference material, your lecture notes, and online resources. You may not share your solution or code with other students or collaborate with other students, receive code from another student, or submit AI-generated code as your own. Your ability to explain your design choices will be taken into account during the oral exam.

Background — Mission Ares IV

You are a firmware engineer at the Ares IV Habitat Module, a crewed deep-space station orbiting Europa. The habitat's life-support system continuously monitors four environmental sensors. Your team has designed a dedicated chip called the Autonomous Crew Protection Unit (ACPU). The chip samples these sensors and asserts hardware alert lines when safety thresholds are violated.

This exam has two parts:

- Part 1: You design the ACPU chip — its threshold logic, truth table, K-maps, and gate-level circuit.
- Part 2: You write the MIPS assembly firmware that reads the chip's live outputs and displays a real-time status panel for the crew.

The two parts are directly connected: the circuit you design in Part 1 is exactly what the chip stub simulates in Part 2. A correct Part 1 design should produce the same fault and alert values you observe in Part 2.

Part 1 — Design the ACPU Chip

1.1 Sensor Inputs and Fault Thresholds

Every clock tick, the ACPU chip reads four sensor values. Each sensor value is an unsigned 8-bit integer (0–255) and located in a separate register (these are your chip inputs). The chip compares each reading to a fixed safety threshold and warns if a value is dangerous:

Fault bit	Sensor	TRUE (warn) when
Fo	Oxygen level	reading < 50
Fp	Pressure level	reading < 80
Ft	Temperature level	reading < 60
Fr	Radiation level	reading > 180

1.2 Alert Output Lines

From the four fault bits, the chip drives three alert output signals using combinational logic. These are the outputs of your chip:

Output	Name	Rule
A	ALARM	Any fault bit is active, EXCEPT when radiation is the ONLY fault bit active
E	EVACUATE	Two or more faults are active simultaneously
L	LOG	Any fault is active, no exceptions

1.3 Questions

1.3.1 Threshold Logic Explanation

In 3–5 sentences, explain how the chip converts a raw 8-bit uint sensor reading into a single binary fault bit. You do not need to design the comparator circuit. Just explain the concept clearly, as if writing a design note for a new team member who will maintain this chip.

1.3.2 Truth Table

Using Fo, Fp, Ft, Fr as the four inputs, construct the full truth table with outputs A, E, L.

- All 16 rows must appear. Do not skip any.
  - The binary values of the inputs should be in numerical order (ie, 0000, 0001, …)

1.3.3 K-Map Minimization

For each of the three outputs (A, E, L), draw a separate 4-variable K-map.

- Write the final minimized Sum-of-Products (SOP) expression beneath each map.
  - Show all of your work. You may use composite gates.

1.3.4 Gate-Level Circuit Diagram

Draw the complete gate-level circuit that implements all three minimized SOP expressions. Use only AND, OR, and NOT gates.

- The four inputs Fo, Fp, Ft, Fr must enter from the left.
- The three outputs A, E, L must exit on the right, clearly labeled.
- Label every gate and every wire.
- Where the same sub-expression appears in more than one output's expression, you may share the gate output — mark shared nodes with a dot (•) and label them.
- The circuit must match your **minimized** expressions exactly.
- It is recommended to use a full page and use colors.

Part 2 — MIPS Firmware

2.1 Context

Your ACPU chip is now connected to the habitat's MIPS computer via a memory-mapped I/O (MMIO) bus. The chip writes live sensor readings and computed fault/alert flags into fixed byte addresses every tick. Your MIPS program reads these addresses in a continuous polling loop and redraws the crew status panel on every iteration.

You are provided with two files:

- acpu_stub.asm — simulates the ACPU chip. **Do not modify this file.**
- student_code.asm — your starting point. **Write all your code here.**

Running in MARS

1. Place both .asm files in the same folder.

2. In MARS: File → Open, then File → Assemble all files in directory.

3. Use the Run Speed slider (top of screen when running code) to slow execution to 30 instructions a second so you can read the display as it updates.

2.2 MMIO Register Map

Address	Name	Contents
0xFFFF0010	O2_LEVEL	Oxygen reading, 0–255
0xFFFF0011	PRESSURE_LEVEL	Pressure reading, 0–255
0xFFFF0012	TEMP_LEVEL	Temperature reading, 0–255
0xFFFF0013	RADIATION_LEVEL	Radiation reading, 0–255
0xFFFF0014	FAULT_FLAGS	Bits 3–0: Fr Ft Fp Fo. Bit = 1 means fault active.
0xFFFF0015	ALERT_FLAGS	Bits 2–0: L E A. Bit = 1 means alert asserted.

- All values are single unsigned bytes. Use lbu (load byte unsigned) to read them — never lw.
- FAULT_FLAGS and ALERT_FLAGS are computed by the chip hardware and reflect your Part 1 circuit. Your firmware reads them; it does not recompute them.

2.3 Program Structure

Your main loop calls the stub to advance sensor data, then calls your display procedure, and repeats forever. The scaffold already provides this — **do not change main.**

```
main:
    jal update_sensors   # advance stub to next reading
    jal print_status     # redraw crew display
    j   main             # poll forever
```

2.4 Required Procedures

You must implement the three procedures below inside student_code.asm. Each must begin with a comment block stating its name, input registers, output registers, and a one-sentence description.

Restrictions that apply to all procedures:

- No mul, mult, div, or divu instructions anywhere.
- Every lbu from an MMIO address must have an inline comment identifying which register is being read.
- You must implement register spilling and use the stack correctly where necessary.

Procedure 1: push_o2_sample

Maintains a circular buffer of the last 8 O2 readings, later it will be used to compute a moving average. A circular buffer simply stores the next item at i+1 and then loops back to index 0 when i reaches the end of the buffer (overwriting the oldest data).

Input	$a0 = new O2 byte (0–255)
Output	none

Data structures — already declared in the scaffold:

```
.data
o2_buf:    .byte 0:8   # 8 one-byte slots
buf_idx:   .word 0     # next write position, 0-7
buf_count: .word 0     # valid entries so far, caps at 8
```

Logic to implement:

- Load buf_idx. Store $a0 as a byte at o2_buf + buf_idx using sb.
- Advance the index: buf_idx = (buf_idx + 1), and reset to 0 if end of buffer has been reached.
- If buf_count < 8, increment buf_count. Otherwise leave it at 8.
- Write the updated index and count back to static memory (via the labels).

Procedure 2: compute_o2_avg

Computes the moving average over the circular buffer using a right-shift instead of division.

Input	none
Output	$v0 = average (0–255), or -1 if fewer than 8 samples have been collected

Logic to implement:

- If the buffer has less than 8 items in it, return -1 (hint, use buf_count)
- If the buffer has 8 items then calculate their average:
  - When computing the average you may not use div or similar instructions for math. You may only use add and the srl (shift right logical) instructions.
- Return the result in $v0.

Procedure 3: print_status

Reads all six MMIO addresses, calls push_o2_sample and compute_o2_avg, and prints the crew status panel to the MARS console.

Input	none
Output	none (prints to MARS console)

Steps to implement:

- Clear the console by printing a newline char “\n” .
- Read O2_LEVEL with lbu. Pass to push_o2_sample.
- Call compute_o2_avg. Save the result ($v0).
- Read PRESSURE_LEVEL, TEMP_LEVEL, RADIATION_LEVEL, FAULT_FLAGS, ALERT_FLAGS with lbu.
- Print the status panel in the format shown below.
- If EVACUATE (bit 1 of ALERT_FLAGS) is set, print an evacuation warning banner after the panel.

Required output format:

```
========================================
   ARES IV -- HABITAT STATUS
========================================
   O2        :  212   [OK]
   Pressure  :   74   [FAULT]
   Temp      :  190   [OK]
   Radiation :   22   [OK]
----------------------------------------
   ALARM     :  NO
   EVACUATE  :  NO
   LOG       :  YES -- event recorded
----------------------------------------
   O2 avg(8) :  198
========================================
```

Display rules:

- For each sensor, extract its fault bit from FAULT_FLAGS using shift and AND. Print [OK] or [FAULT].
- For ALARM, EVACUATE, LOG: extract each bit from ALERT_FLAGS. Print YES or NO.
- If ALARM is YES, also print which sensor(s) caused it on the line(s) below.
- If EVACUATE is YES, print "EVACUATE -- CREW TO ESCAPE PODS" after the panel.
- If the O2 average is -1, print not enough data instead of a number.

Grading

Area	Points
Part 1 — Threshold explanation	5
Part 1 — Truth table	5
Part 1 — K-maps	10
Part 1 — Circuit	25
Part 2 — Circular buffer	5
Part 2 — Moving average	10
Part 2 — Status display	25
Part 2 — Runs in MARS	10
Overall — Comments and Clarity	5